POO

Programação Orientada a Objetos com PHP

Introdução à Programação Orientada a Objetos em PHP

1. O que é Programação Orientada a Objetos?

A Programação Orientada a Objetos (POO) é um **paradigma de programação**, ou seja, uma forma de pensar e organizar o código, baseado no conceito de **objetos**. Um objeto é uma entidade que possui **características** (dados) e **comportamentos** (ações).

Antes da POO, o modelo predominante era a **programação procedural**, onde o código era escrito como uma sequência linear de instruções e funções soltas. À medida que os sistemas cresciam, esse modelo se tornava difícil de manter, entender e escalar.

A POO surgiu como resposta a esse problema, trazendo uma forma de modelar o software de maneira muito mais próxima do mundo real.

Analogia: Pense em um carro. Todo carro tem **atributos** (cor, marca, modelo, velocidade atual) e **comportamentos** (acelerar, frear, ligar, desligar). Na POO, modelamos isso exatamente da mesma forma: criamos uma "planta" chamada **Classe** que descreve como um carro deve ser, e a partir dela fabricamos quantos **objetos** (instâncias) quisermos.

Os 4 Pilares da POO

Pilar	Descrição
Encapsulamento	Proteger os dados internos de um objeto, expondo apenas o necessário
Herança	Uma classe pode herdar características de outra classe
Polimorfismo	Um mesmo método pode se comportar de formas diferentes conforme o contexto
Abstração	Modelar apenas os aspectos relevantes de uma entidade para o sistema

Ao longo do curso, exploraremos cada um desses pilares em detalhes. Nesta aula, daremos o primeiro passo.

2. Por que a POO é importante?

A adoção da POO não é apenas uma preferência estética. Ela resolve problemas reais que surgem em projetos de médio e grande porte. Veja os principais benefícios:

Organização e clareza: O código fica dividido em unidades lógicas e coesas. Em vez de um arquivo com centenas de funções soltas, temos classes com responsabilidades bem definidas.

Reutilização de código: Uma classe bem escrita pode ser reutilizada em múltiplos projetos. O mecanismo de herança potencializa ainda mais essa capacidade.

Manutenibilidade: Quando um sistema cresce, a POO permite localizar e corrigir problemas com muito mais facilidade. Você sabe exatamente onde cada responsabilidade está.

Trabalho em equipe: Diferentes desenvolvedores podem trabalhar em classes distintas simultaneamente sem que o código de um interfira diretamente no do outro.

Base para frameworks modernos: Todo o ecossistema PHP moderno, Laravel, Symfony, CodeIgniter, é construído sobre POO. Dominar esse paradigma é pré-requisito para trabalhar com qualquer framework profissional.

3. Conceitos Fundamentais: Classe e Objeto

Antes de ver o código, é essencial fixar dois conceitos centrais:

Classe é o molde, a planta, o projeto. Ela define quais atributos e métodos os objetos desse tipo terão. A classe não ocupa memória de dados, ela é apenas uma definição.

Objeto é uma instância concreta de uma classe. Quando criamos um objeto, a memória é alocada e aquele objeto passa a ter sua própria cópia dos atributos definidos na classe.

```
Classe Carro   →  objeto $carro1 (Honda Civic, Preto)
               →  objeto $carro2 (Toyota Corolla, Branco)
               →  objeto $carro3 (Ford Mustang, Vermelho)
```

Três objetos diferentes, todos criados a partir da mesma classe, cada um com seus próprios valores de atributos.

4. Diagramas de Classe e Código PHP

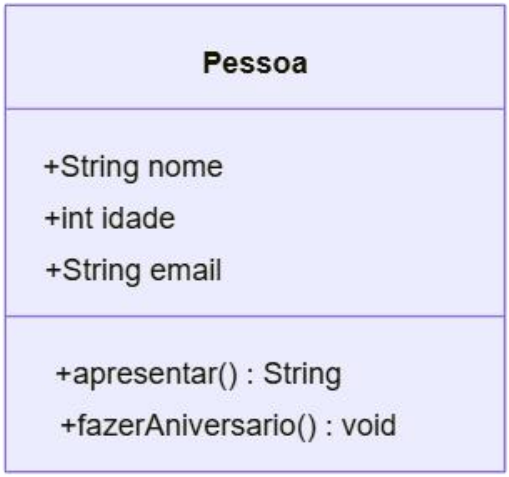
O **Diagrama de Classes** é uma ferramenta da UML (Unified Modeling Language) usada para representar visualmente a estrutura das classes de um sistema. Ele é fundamental para planejar e comunicar a arquitetura antes de escrever uma linha de código.

Estrutura básica de um Diagrama de Classes

```
+---------------------------+
|      NomeDaClasse         |  ← Nome da classe
+---------------------------+
| + atributo1 : tipo        |  ← Atributos (+ = público)
| + atributo2 : tipo        |
+---------------------------+
| + metodo1() : retorno     |  ← Métodos
| + metodo2() : retorno     |
+---------------------------+
```

O símbolo **+** indica visibilidade **pública**. Veremos outros modificadores em breve.

Exemplo 1: Diagrama da classe Pessoa

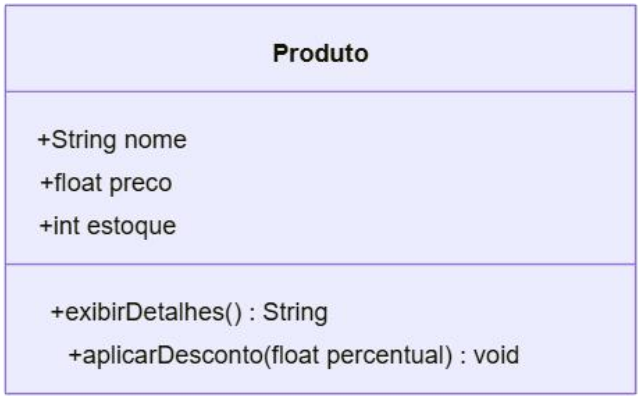


Código PHP

```
<?php

class Pessoa {

    public string $nome;
    public int $idade;
    public string $email;

    public function apresentar()
    {
        return "Olá! Meu nome é {$this->nome}, tenho {$this->idade} anos.";
    }

    public function fazerAniversario()
    {
        $this->idade++;
        echo "Feliz aniversário, {$this->nome}! Agora você tem {$this->idade}
         anos.\n";
    }
}

// Criando objetos (instâncias) da classe Pessoa
$pessoa1 = new Pessoa();
$pessoa1->nome  = "Ana Silva";
$pessoa1->idade = 28;
$pessoa1->email = "ana@email.com";

$pessoa2 = new Pessoa();
```

```
$pessoa2->nome  = "Carlos Souza";
$pessoa2->idade = 35;
$pessoa2->email = "carlos@email.com";

// Utilizando os métodos
echo $pessoa1->apresentar() . "\n";
// Saída: Olá! Meu nome é Ana Silva, tenho 28 anos.

$pessoa2->fazerAniversario();
// Saída: Feliz aniversário, Carlos Souza! Agora você tem 36 anos.
```

Exemplo 2: Diagrama da classe Produto

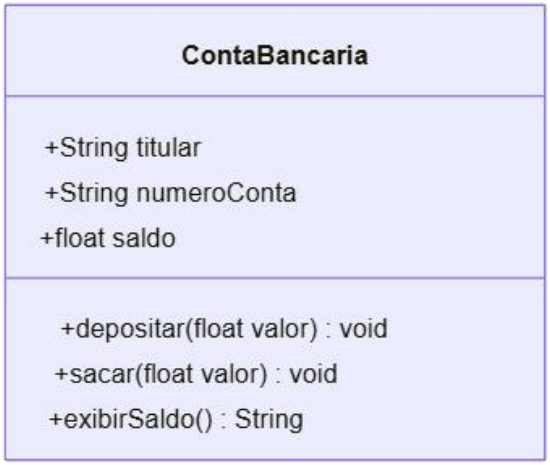


Código PHP

```
<?php

class Produto {

    public string $nome;
    public float  $preco;
    public int    $estoque;

    public function exibirDetalhes() {
        return "Produto: {$this->nome} | "
            . "Preço: R$ " . number_format($this->preco, 2, ',', '.') . " | "
            . "Estoque: {$this->estoque} unidades";
    }

    public function aplicarDesconto(float $percentual)   {
        $desconto      = $this->preco * ($percentual / 100);
        $this->preco  -= $desconto;
        echo "Desconto de {$percentual}% aplicado! Novo preço: R$ "
           . number_format($this->preco, 2, ',', '.') . "\n";
    }
}
```

```
// Criando e utilizando objetos
$produto1 = new Produto();
$produto1->nome    = "Notebook Gamer";
$produto1->preco   = 4500.00;
$produto1->estoque = 10;

$produto2 = new Produto();
$produto2->nome    = "Mouse Sem Fio";
$produto2->preco   = 150.00;
$produto2->estoque = 50;

echo $produto1->exibirDetalhes() . "\n";
// Saída: Produto: Notebook Gamer | Preço: R$ 4.500,00 | Estoque: 10
unidades

$produto1->aplicarDesconto(10);
// Saída: Desconto de 10% aplicado! Novo preço: R$ 4.050,00

echo $produto2->exibirDetalhes() . "\n";
// Saída: Produto: Mouse Sem Fio | Preço: R$ 150,00 | Estoque: 50
unidades
```

Exemplo 3: Diagrama da classe ContaBancaria



Código PHP

```
<?php

class ContaBancaria
{
    public string $titular;
    public string $numeroConta;
```

```
    public float  $saldo;

    public function depositar(float $valor): void
    {
        $this->saldo += $valor;
        echo "Depósito de R$ " . number_format($valor, 2, ',', '.')
           . " realizado. Saldo atual: R$ "
           . number_format($this->saldo, 2, ',', '.') . "\n";
    }

    public function sacar(float $valor): void
    {
        if ($valor > $this->saldo) {
            echo "Saldo insuficiente para o saque de R$ "
               . number_format($valor, 2, ',', '.') . "\n";
            return;
        }
        $this->saldo -= $valor;
        echo "Saque de R$ " . number_format($valor, 2, ',', '.')
           . " realizado. Saldo atual: R$ "
           . number_format($this->saldo, 2, ',', '.') . "\n";
    }

    public function exibirSaldo(): string
    {
        return "Titular: {$this->titular} | Conta: {$this->numeroConta} | "
             . "Saldo: R$ " . number_format($this->saldo, 2, ',', '.');
    }
}

// Criando e operando uma conta bancária
$conta = new ContaBancaria();
$conta->titular     = "João Pereira";
$conta->numeroConta = "12345-6";
$conta->saldo       = 1000.00;

echo $conta->exibirSaldo() . "\n";
// Saída: Titular: João Pereira | Conta: 12345-6 | Saldo: R$ 1.000,00

$conta->depositar(500.00);
// Saída: Depósito de R$ 500,00 realizado. Saldo atual: R$ 1.500,00

$conta->sacar(200.00);
// Saída: Saque de R$ 200,00 realizado. Saldo atual: R$ 1.300,00

$conta->sacar(2000.00);
// Saída: Saldo insuficiente para o saque de R$ 2.000,00
```

5. O Problema dos Atributos Públicos

Ao longo dos exemplos desta aula, usamos o modificador public em todos os atributos. Isso torna o desenvolvimento inicial mais simples e direto, o que é ótimo para aprender os conceitos básicos. No entanto, atributos públicos carregam um risco sério em aplicações reais.

Observe o que é possível fazer com a nossa classe ContaBancaria:

```
// Nada impede que qualquer parte do código faça isso:
$conta->saldo = -999999.00;  // Saldo negativo absurdo inserido
diretamente!
$conta->saldo = "isso não é um número";  // Valor inválido no atributo!
$conta->titular = "";  // Titular vazio sem nenhuma validação!
```

Como o atributo é public, **qualquer código em qualquer lugar do sistema pode alterar seu valor diretamente**, sem nenhum tipo de validação ou controle. Isso viola um dos pilares mais importantes da POO que foi mencionado no início: o **Encapsulamento**.

Em um sistema bancário real, isso seria catastrófico. Mas o mesmo risco existe em qualquer sistema: um preço de produto que vira negativo, uma idade que recebe um valor de 9999, um e-mail inválido armazenado sem verificação.

Atividade referente a aula assíncrona do dia 06/05/2026

Como atividade da aula assíncrona do dia 06 de maio, cada aluno deverá elaborar um resumo sobre o conteúdo abordado em sala de aula, de próprio punho e com letra legível. Após a conclusão, é necessário tirar uma foto da folha e anexá-la na atividade disponibilizada na plataforma Teams.

Atividades enviadas fora do prazo não serão aceitas. A presença referente a esse dia estará vinculada à entrega da atividade; portanto, o aluno que não a entregar receberá falta.